

JANUARY 7, 2026

CLOUD INFRASTRUCTURE VALIDATION REPORT

SAWSAN MOUNES (STUDENT)
STUDENT ID:MOU23620003

TABLE OF CONTENTS

1.Executive Summary	2
2.Scope & Objectives.....	2
3.Architecture Overview (London + Frankfurt).....	3
3.1 Networking	3
3.2 Compute, Load Balancing & Scaling	4
3.3 Storage & Encryption.....	5
4.Implementation Steps (Terraform + AWS).....	6
4.1 How my Terraform was organised	6
4.2 What I did to deploy	6
4.3 Why multi-region	7
5.Validation & Evidence.....	7
5.1 Terraform Verification.....	7
5.2 AWS CLI Validation.....	8
6.IAM Roles & Groups Justification (least privilege).....	8
6.1 IAM Groups Created	8
6.2 What Each Team Can Do (and Why)	8
6.3 Replication Role (Service Role)	9
7.Challenges Encountered & Fixes.....	10
8.Security Controls	10
9.Recommendations / Improvements	11
10.Conclusion.....	11

1.EXECUTIVE SUMMARY

This report reviews the AWS cloud infrastructure I built using Terraform for a global organisation. It focuses on a secure multi-region setup and checks that the IAM roles and team permissions I created match what each team NEEDS. The design was built around the coursework scenario: supporting a digital-transformation provider with secure cloud migration, automation, monitoring, and safe handling of customer data. I explain what I deployed, how I validated it, the main problems I ran into (and how I fixed them), and the recommendations I would make to improve security and resilience.

The solution uses London (eu-west-2) as the primary region and Frankfurt (eu-central-1) as the secondary region. Both regions are deployed with separate VPCs and a split network design (public and private subnets). Public workloads route through an Internet Gateway, while private workloads use a NAT Gateway for outbound access without being exposed to the internet. The application is hosted behind an Application Load Balancer (ALB) and scaled using an Auto Scaling Group (ASG) with scale-in and scale-out policies. Customer records are stored in an encrypted S3 bucket in London and replicated to Frankfurt using S3 replication and KMS multi-region keys to keep data protected in both regions.

Key deployment outputs:

- London ALB DNS: cw-london-alb-1290824451.eu-west-2.elb.amazonaws.com
- Frankfurt ALB DNS: cw-frankfurt-alb-1513241870.eu-central-1.elb.amazonaws.com
- ASG capacity (both regions): min=1, desired=2, max=3
- S3 buckets: cw-customer-records-sawsan-001 (London) and cw-customer-records-sawsan-001-frankfurt (Frankfurt)
- KMS multi-region key ARNs (primary + replica) recorded in the Appendix

2.SCOPE & OBJECTIVES

The scope of this project was to build and validate a secure AWS environment using Terraform for a multi-region setup.

Scope :

Multi-region infrastructure in London (eu-west-2) and Frankfurt (eu-central-1).

VPC networking with public/private subnets, routing, IGW and NAT.

ALB + Auto Scaling Group with CloudWatch alarms for monitoring/scaling.

IAM groups/roles/policies for different teams using least privilege.

S3 storage for customer records encrypted with KMS, with replication to Frankfurt.

Objectives :

Build a basic AWS setup using Terraform to match the scenario.

Create a VPC with public + private subnets (Sales in public with IGW, Compliance in private with NAT).

Set up IAM groups and least-privilege EC2 roles for each team (D-Transformation read/write, Sales read-only).

Create security groups per team with strict inbound/outbound rules.

Deploy an ALB + Auto Scaling Group with scale in/out policies.

Store customer data securely using encrypted storage (S3 + KMS) and document everything in a report (steps, issues, fixes, recommendations).

3.ARCHITECTURE OVERVIEW (LONDON + FRANKFURT)

3.1 NETWORKING

London VPC (eu-west-2): I built the main VPC in London using 10.0.0.0/16, split into two public subnets and two private subnets across different Availability Zones.

Frankfurt VPC (eu-central-1): I also created a second VPC in Frankfurt using 10.1.0.0/16, with the same setup (2 public + 2 private subnets) for consistency

Terraform showing CIDR block for both London and Frankfurt :

```
(base) sawsanmounes@Sawsan-2 sawsancyber-TERRAFORM % terraform state show module.london.aws_vpc.this | egrep "cidr_block|tags"
terraform state show module.frankfurt.aws_vpc.this | egrep "cidr_block|tags"

  assign_generated_ipv6_cidr_block = false
  cidr_block                       = "10.0.0.0/16"
  ipv6_cidr_block                 = null
  ipv6_cidr_block_network_border_group = null
  tags                           = {
    tags_all = {
  assign_generated_ipv6_cidr_block = false
  cidr_block                       = "10.1.0.0/16"
  ipv6_cidr_block                 = null
  ipv6_cidr_block_network_border_group = null
  tags                           = {
    tags_all = {
```

Main networking setup:

Each VPC has an Internet Gateway so the public subnets can reach the internet.

A NAT Gateway sits in the public subnet to let private subnet instances (Compliance) access the internet securely without being publicly exposed.

Route tables were configured so:

Public route table: 0.0.0.0/0 → IGW

Private route table: 0.0.0.0/0 → NAT

Frankfurt route table on AWS :

rtb-0d88bdf89633485 / cw-london-private-rt

Detailsinfo

Route table ID

rtb-0d88bdf89633485

VPC

vpc-0c34a0685150004d8 | cw-london-vpc

Main

No

Owner ID

888647520191

Explicit subnet associations

2 subnets

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Filter routes

Destination	Target	Status
0.0.0.0/0	nat-014eb3c9564ba086b	Active
10.0.0.0/16	local	Active

London route table on aws :

rtb-042948267307b844d / cw-london-public-rt			
Details info			
Route table ID	rtb-042948267307b844d	Main	No
VPC	vpc-0c34a0685150004d8 cw-london-vpc	Owner ID	888647520191
Explicit subnet associations		2 subnets	
Routes Subnet associations Edge associations Route propagation Tags			
Routes (2)			
Filter routes			
Destination	Target	Status	
0.0.0.0/0	igw-0fbc63db4e770b9e07c	Active	
10.0.0.0/16	local	Active	

3.2 COMPUTE, LOAD BALANCING & SCALING

3.2 Compute, Load Balancing & Scaling (reworded)

The system runs on EC2 servers in each region. Users don't connect to the servers directly — they connect to an Application Load Balancer (ALB) first. The ALB then sends the traffic to the correct EC2 instances. An Auto Scaling Group (ASG) controls how many EC2 instances are running so the service stays available if demand changes.

ALB DNS names in London and Frankfurt (Terraform state output).

```
(base) sawsanmmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state show module.london.aws_lb.app | grep -n "dns_name"
terraform state show module.frankfurt.aws_lb.app | grep -n "dns_name"

18:      dns_name                                = "cw-london-alb-311923140.eu-west-2.elb.amazonaws.com"
8:      dns_name                                = "cw-frankfurt-alb-1801856980.eu-central-1.elb.amazonaws.co
m"
```

The ALB listens on port 80 (HTTP) and forwards requests to the target group (the EC2 instances).

ALB listener configuration showing HTTP/80 and target group ARNs (Terraform state output).

```
can't command not found: terraform
(base) sawsanmmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state show module.london.aws_lb_listener.http | egrep -n "port|
rotocol|default_action|target_group_arn"

5:      port                                    = 80
7:      protocol                               = "HTTP"
16:     routing_http_response_strict_transport_security_header_value = null
22:     default_action {
24:       target_group_arn = "arn:aws:elasticloadbalancing:eu-west-2:888647520191:targetgroup/cw-london-tg/d149842a4917a00c"
(base) sawsanmmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state show module.frankfurt.aws_lb_listener.http | egrep -n "por
t|protocol|default_action|target_group_arn"

5:      port                                    = 80
7:      protocol                               = "HTTP"
16:     routing_http_response_strict_transport_security_header_value = null
22:     default_action {
24:       target_group_arn = "arn:aws:elasticloadbalancing:eu-central-1:888647520191:targetgroup/cw-frankfurt-tg/7d0f3e7e3ef80
28: ~"
```

Details info

Route table ID

rtb-02486eb5e85c1157f

VPC

vpc-0f36489837f4b058f | cw-frankfurt-vpc

Main

No

Owner ID

888647520191

Explicit

2 subnets

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Filter routes

Destination

▼

Target

▼

Status

0.0.0.0/0

[igw-0419abe20c0db615c6](#)

Active

10.1.0.0/16

local

Active

Detailsinfo

Route table ID

rtb-08ac46fc8e2b1fed8

VPC

vpc-0f36489837f4b058f | cw-frankfurt-vpc

Main

No

Owner ID

888647520191

Ex 2

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Filter routes

Destination

▼

Target

▼

Status

0.0.0.0/0

nat-0278125db691687fd

Active

10.1.0.0/16

local

Active

Target group settings (HTTP/80) for both regions (Terraform state output).

```
(base) sawsanmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state show module.london.aws_lb_target_group.app | egrep -n "name|port|protocol"
13:   name           = "cw-london-tg"
14:   name_prefix    = null
15:   port           = 80
16:   protocol       = "HTTP"
17:   protocol_version = "HTTP1"
18:   proxy_protocol_v2 = false
35:     port         = "traffic-port"
36:     protocol     = "HTTP"
43:     cookie_name  = null
(base) sawsanmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state show module.frankfurt.aws_lb_target_group.app | egrep -n "name|port|protocol"
13:   name           = "cw-frankfurt-tg"
14:   name_prefix    = null
15:   port           = 80
16:   protocol       = "HTTP"
17:   protocol_version = "HTTP1"
18:   proxy_protocol_v2 = false
35:     port         = "traffic-port"
36:     protocol     = "HTTP"
```

The ASG keeps 2 instances running normally (desired = 2), but it can go down to 1 (min = 1) or up to 3 (max = 3).

Auto Scaling Group capacity values in both regions (Terraform state output).

```
(base) sawsanmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state show module.london.aws_autoscaling_group.app | egrep -n "min_size|max_size|desired_capacity"
terraform state show module.frankfurt.aws_autoscaling_group.app | egrep -n "min_size|max_size|desired_capacity"
12:   desired_capacity      = 2
13:   desired_capacity_type = null
23:   max_size              = 3
25:   min_size              = 1
12:   desired_capacity      = 2
13:   desired_capacity_type = null
23:   max_size              = 3
25:   min_size              = 1
```

CloudWatch alarms + scaling rules automatically add an instance when load is high (scale out) and remove one when load is low (scale in).

3.3 STORAGE & ENCRYPTION

Customer records are saved in an S3 bucket in London and it's locked down using SSE-KMS encryption (so AWS encrypts everything with our KMS key). I also enabled bucket versioning so files can be recovered and so replication works properly.

customer records buckets + KMS key ARNs

```
(base) sawsanmounes@sawsan-2 sawsancyber-TERRAFORM % terraform output | egrep "customer_records"
customer_records_bucket_frankfurt = "cw-customer-records-sawsan-001-frankfurt"
customer_records_bucket_name      = "cw-customer-records-sawsan-001"
customer_records_kms_key_arn      = "arn:aws:kms:eu-west-2:888647520191:key/mrk-701ba11225a04a59866ef1b717704884"
customer_records_kms_replica_key_arn_frankfurt = "arn:aws:kms:eu-central-1:888647520191:key/mrk-701ba11225a04a59866ef1b717704884"
```

To make the data more resilient, I set up cross-region replication from London to Frankfurt. The replicated objects stay encrypted, and the Frankfurt bucket uses the KMS multi-region replica key.

S3 replication configuration — London → Frankfurt (SSE-KMS objects only)

```
(base) sawsanmounes@Sawsan-2 sawsancyber-TERRAFORM % aws s3api get-bucket-replication --bucket cw-customer-records-sawsan-001
{
  "ReplicationConfiguration": {
    "Role": "arn:aws:iam::888647520191:role/cw-s3-replication-role",
    "Rules": [
      {
        "ID": "london-to-frankfurt",
        "Priority": 0,
        "Filter": {},
        "Status": "Enabled",
        "SourceSelectionCriteria": {
          "SseKmsEncryptedObjects": {
            "Status": "Enabled"
          }
        },
        "Destination": {
          "Bucket": "arn:aws:s3:::cw-customer-records-sawsan-001-frankfurt",
          "StorageClass": "STANDARD",
          "EncryptionConfiguration": {
            "ReplicaKmsKeyID": "arn:aws:kms:eu-central-1:888647520191:key/mrk-701ba11225a04a59866ef1b717704884"
          }
        },
        "DeleteMarkerReplication": {
          "Status": "Enabled"
        }
      }
    ]
  }
}
```

- London S3 bucket: cw-customer-records-sawsan-001
- Frankfurt S3 bucket: cw-customer-records-sawsan-001-frankfurt
- KMS multi-region key: primary in eu-west-2 + replica in eu-central-1 (same MRK ID)
- Replication rule: london-to-frankfurt (SSE-KMS encrypted objects only + delete markers replicated)

KMS customer records key — multi-region enabled + key rotation on

```
(base) sawsanmounes@Sawsan-2 sawsancyber-TERRAFORM % terraform state show aws_kms_key.customer_records | egrep -n "arn|multi_re
gion|enable_key_rotation"

3:      arn                                = "arn:aws:kms:eu-west-2:888647520191:key/mrk-701ba11225a04a59866ef1b717704884"
8:      enable_key_rotation                = true
13:     multi_region                      = true
22:                                     AWS = "arn:aws:iam::888647520191:root"
```

4.IMPLEMENTATION STEPS (TERRAFORM + AWS)

4.1 HOW MY TERRAFORM WAS ORGANISED

- I used two AWS regions: London (eu-west-2) and Frankfurt (eu-central-1) using provider aliases.
- I used a reusable module (modules/vpc_stack) to build the main network + compute stuff (VPC, subnets, security groups, ALB, ASG) for each region.
- Then I kept storage + IAM + KMS + replication in the root files.

4.2 WHAT I DID TO DEPLOY

- terraform init
- terraform fmt -recursive
- terraform validate
- terraform plan
- terraform apply

- After that, I checked everything using:
 - terraform output
 - terraform state show
 - AWS CLI commands (like checking replication)

4.3 WHY MULTI-REGION

- London is the main region, Frankfurt is the backup.
- If London fails, the design still has a second region ready, and customer data is replicated to Frankfurt for resilience.

5. VALIDATION & EVIDENCE

I validated the build using Terraform outputs/state and AWS CLI checks to prove the key security and resilience controls were actually in place.

5.1 TERRAFORM VERIFICATION

Evidence collected from Terraform included:

- terraform output
(confirmed S3 bucket names + KMS key ARNs)
- terraform state show module.<region>.aws_lb.app | grep dns_name
(confirmed the ALB DNS names for London + Frankfurt)
- terraform state show module.<region>.aws_autoscaling_group.app | egrep "min_size|max_size|desired_capacity"
(confirmed ASG scaling values: min/desired/max)
- terraform state list | grep aws_iam_
(confirmed IAM groups, policies, and roles were created)

example iam groups, policies, and roles created

```
(base) sawsanmounes@sawsan-2 sawsancyber-TERRAFORM % terraform state list | egrep "aws_iam_group|aws_iam_role|aws_iam_policy" | head -n 30
data.aws_iam_policy_document.customer_records_bucket_policy
data.aws_iam_policy_document.ec2_assume_role
data.aws_iam_policy_document.kms_customer_records_base
data.aws_iam_policy_document.s3_replication_assume_role
data.aws_iam_policy_document.s3_replication_policy
aws_iam_group.teams["AdministrativeTeam"]
aws_iam_group.teams["ComplianceTeam"]
aws_iam_group.teams["D-TransformationTeam"]
aws_iam_group.teams["DevOpsTeam"]
aws_iam_group.teams["RedTeam"]
aws_iam_group.teams["SalesTeam"]
aws_iam_group_policy_attachment.admin_adminaccess
aws_iam_group_policy_attachment.compliance_group_audit
aws_iam_group_policy_attachment.devops_group_cloudwatch
aws_iam_group_policy_attachment.dtrans_group_readwrite
aws_iam_group_policy_attachment.redteam_group_readonly
aws_iam_group_policy_attachment.sales_group_readonly
aws_iam_policy.compliance_audit_readonly
aws_iam_policy.devops_cloudwatch_readonly
aws_iam_policy.dtrans_s3_readwrite
aws_iam_policy.redteam_readonly_visibility
aws_iam_policy.s3_replication
aws_iam_policy.sales_s3_readonly
aws_iam_role.compliance_ec2_role
aws_iam_role.s3_replication
aws_iam_role.sales_ec2_role
aws_iam_role_policy_attachment.compliance_ec2_attach_readonly
aws_iam_role_policy_attachment.s3_replication_attach
```


5.2 AWS CLI VALIDATION

Key AWS CLI checks I ran:

```
aws s3api get-bucket-encryption --bucket cw-customer-records-sawsan-001
aws s3api get-bucket-versioning --bucket cw-customer-records-sawsan-001
aws s3api get-bucket-replication --bucket cw-customer-records-sawsan-001
aws iam list-groups
aws iam list-policies --scope Local | egrep "cw-"
```

The AWS replication output confirmed the replication rule (london-to-frankfurt) was enabled, and that replication only applies to KMS-encrypted objects, using the Frankfurt replica KMS key ARN.

6.IAM ROLES & GROUPS JUSTIFICATION (LEAST PRIVILEGE)

For IAM, I followed least privilege so each team only has the access they actually need for their job. Instead of giving everyone broad permissions, I created one IAM group per team and attached either an AWS managed policy or a custom policy limited to specific services (and in some cases, specific resources like the customer records S3 bucket).

Where EC2 needed AWS access, I used IAM instance roles (so no access keys were stored on the servers).

6.1 IAM GROUPS CREATED

- AdministrativeTeam
- Compliance Team
- DevOpsTeam
- D-TransformationTeam
- RedTeam
- SalesTeam

6.2 WHAT EACH TEAM CAN DO (AND WHY)

AdministrativeTeam — Full admin

- Access: AdministratorAccess (AWS managed)
- Why: This team needs full control for admin tasks like account management. It's limited to this group only.

ComplianceTeam — Audit + evidence (read-only)

- Access: cw-compliance-audit-readonly + EC2 role cw-compliance-ec2-role
- What they can do: view CloudTrail/audit logs and read customer records (no edits)

- Why: Compliance should be able to gather evidence and check logs without being able to change infrastructure or data.

DevOpsTeam — Monitoring only

- Access: cw-devops-cloudwatch-readonly
- What they can do: view CloudWatch metrics/logs and monitoring information
- Why: DevOps needs visibility to monitor the system, but read-only prevents accidental config changes.

D-TransformationTeam — Work with customer records (restricted read/write)

- Access: cw-dtrans-s3-readwrite
- What they can do: read/write/delete objects only inside the customer records S3 bucket
- Why: They upload and process customer files, so they need write access — but it's limited to the approved bucket only.

RedTeam — Visibility for testing (no changes)

- Access: cw-redteam-readonly-visibility
- What they can do: describe/list/read-only actions for assessment
- Why: RedTeam needs enough access to validate security and configuration, but no permissions to modify anything.

SalesTeam — Read customer docs only

- Access: cw-sales-s3-readonly + EC2 role cw-sales-ec2-role
- What they can do: list bucket + read objects only (no upload/delete)
- Why: Sales should only view customer records. Read-only helps protect the integrity of stored documents.

6.3 REPLICATION ROLE (SERVICE ROLE)

A separate IAM role was also created for replication:

cw-s3-replication-role

- Purpose: allows Amazon S3 to replicate objects from the London bucket to the Frankfurt bucket
- Includes: permissions to read from the source bucket, write to the destination bucket, and use KMS (decrypt in London / encrypt in Frankfurt)
- Why: replication needs its own controlled permissions, and it should not depend on any human user account.

7. CHALLENGES ENCOUNTERED & FIXES

During the build and testing, I ran into a few issues. These were fixed as part of the final setup:

1) S3 replication + SSE-KMS error

- What happened: Replication failed because the bucket was encrypted with KMS, but the replication rule didn't fully specify that only KMS-encrypted objects should be replicated.
- Fix: I added `SourceSelectionCriteria` with `SseKmsEncryptedObjects = Enabled`, and made sure the destination used the Frankfurt replica KMS key ARN in the encryption configuration.

2) "No outputs found" when running terraform output

- What happened: Terraform showed no outputs because there was no active state yet (for example, before doing apply, or after destroying).
- Fix: I ran terraform apply again so the resources and state existed, and then re-ran terraform output.

3) Destroy failed because of replication + versioning

- What happened: When destroying, Terraform tried to change bucket versioning, but AWS blocked it because replication was still enabled. This caused an `InvalidBucketState` error.
- Fix: I removed the replication configuration first (using `aws s3api delete-bucket-replication` or by removing replication from Terraform), then versioning changes and bucket deletion worked.

8. SECURITY CONTROLS

To keep the setup secure, I added controls across networking, IAM, compute, and storage:

- Network separation: public and private subnets were used, with an Internet Gateway for public access and a NAT Gateway for private outbound access.
- Security groups per team: each team had its own security group with strict rules to reduce unnecessary access.
- Least privilege IAM: each team got the minimum permissions needed; EC2 used roles instead of access keys.
- Secure S3 storage: customer records are encrypted using SSE-KMS, and bucket policies enforce encryption and secure transport.

- Cross-region replication: customer data is replicated from London to Frankfurt to improve resilience.
- Availability + scaling: the ALB + ASG improves uptime and supports automatic recovery if an instance fails.

9. RECOMMENDATIONS / IMPROVEMENTS

If this was going into a real production environment, these are the main upgrades I would add next:

- Add HTTPS (443) on the ALB with an ACM certificate and redirect HTTP → HTTPS.
- Put AWS WAF in front of the ALB to help block common attacks and limit abusive traffic.
- Enable stronger detection and compliance tools like CloudTrail (organisation-wide), AWS Config, and GuardDuty.
- Use VPC endpoints (S3 Gateway endpoint + interface endpoints like KMS/SSM) so private instances don't rely on the public internet.
- Enable S3 access logging and add lifecycle rules (e.g., move older data to Glacier to reduce cost).
- Treat replication as resilience, not backup — add a real backup/restore plan and test recovery.
- Use SSM Session Manager instead of relying on SSH access.
- Improve hardening: use updated AMIs, patch management, and vulnerability scanning.

10. CONCLUSION

The Terraform build covered what the coursework asked for, and it resulted in a secure multi-region AWS setup. I used two regions with separate public and private networking, team-based security groups, least-privilege IAM groups/roles, and a compute layer using an Application Load Balancer and Auto Scaling. Customer records are stored in S3 with SSE-KMS encryption and replicated to a second region to improve resilience if one region has issues.

I checked the deployment using Terraform outputs/state and AWS CLI commands to confirm the main components were created and configured correctly. The recommendations above are the next steps I'd take to make the setup more secure and closer to a real production standard.